

FlowGuard: A Graph-Based, Temporal, and Resource-Efficient Anti-Money Laundering Detection System

The global financial ecosystem processes unprecedented volumes of transactions daily, characterized by real-time execution and high-velocity cross-border flows. Traditional Anti-Money Laundering (AML) systems are increasingly incapable of detecting sophisticated financial crimes within this environment. Legacy architectures predominantly rely on fixed transaction-amount thresholds, rigid rule-based alerts, and the analysis of individual transactions in isolation¹. These systems fail to capture the multi-hop, structural nature of money laundering, where illicit actors obscure the origin of funds through a sequence of distributed transfers across numerous accounts. A fund movement starting at Account A with ₹10,00,000 may pass through Accounts B and C, shedding minimal amounts, before aggregating at Account D with ₹9,50,000². Individually, these transfers evade static detection rules; structurally, they form a highly suspicious scatter-gather or layering topology. The proposed FlowGuard system aims to address this systemic vulnerability by modeling financial transactions as a Continuous-Time Dynamic Graph (CTDG)⁵. By extracting graph-based and temporal behavioral features, the system utilizes advanced gradient-boosted machine learning models to identify complex laundering topologies. The fundamental research objective is to develop a reproducible, graph-based AML detection pipeline, evaluating how the integration of temporal motifs, value-flow dynamics, and adaptive graph exploration improves upon established graph-feature baselines, while strictly maintaining acceptable processing times, memory efficiency, and low false-positive rates⁷.

Foundational Parameters and Data Acquisition Strategy

To ensure empirical rigor and prevent unfair algorithmic comparisons, the initial phase of the research mandates the freezing of experimental parameters. Every subsequent model iteration must utilize identical datasets, prediction targets, temporal data splits, and evaluation procedures. The environment standardizes the Python version, specific hardware configurations (CPU, RAM, GPU), and critical library versions (e.g., XGBoost, Snap ML, NetworkX).

Selecting the primary experimental dataset requires careful verification of data schemas and ground-truth labels. The research primarily utilizes the IBM synthetic AML datasets, specifically the High-Illicit (AML-HI) and Low-Illicit (AML-LI) variants, alongside real-world benchmarks such as the Ethereum Phishing dataset⁹. Prior to deployment, dataset compatibility is verified to ensure the presence of mandatory fields: unique transaction identifiers, timestamps, source and destination accounts, transaction amounts, and definitive laundering labels.

Dataset Variant	Nodes (Accounts)	Edges (Transactions)	Illicit Rate	Temporal Span
AML HI Small	0.5 M	5 M	0.102%	10 days
AML HI Medium	2.1 M	32 M	0.110%	16 days
AML HI Large	2.1 M	180 M	0.124%	97 days
AML LI Small	0.7 M	7 M	0.051%	10 days
ETH Phishing (Real)	2.9 M	13 M	0.278%	1261 days

The ingestion framework enforces an immutable raw data layer. Incoming datasets undergo cryptographic checksum validation, ensuring that original files are never overwritten. Subsequent cleaning and transformation processes generate new artifacts in isolated processing directories. This structured ingestion captures comprehensive metadata, recording dataset versions, download dates, SHA-256 checksums, and generation configurations, ensuring absolute reproducibility across all experimental phases.

Data Quality, Validation, and Canonical Modeling

Before topological graph construction, an exhaustive Exploratory Data Analysis (EDA) profiles the transactional and account-level distributions. Transaction-level metrics include aggregate volumes, amount percentiles, missing value ratios, and the specific ratio of suspicious to normal activity. Account-level metrics quantify in-degree and out-degree distributions, identifying high-velocity hubs and dormant accounts suddenly exhibiting burst activity—common indicators in mule-account fan-outs².

Because raw AML datasets originate from disparate banking cores, a canonical data schema normalizes all records.

Canonical Field	Data Type	Description and Purpose
transaction_id	String	Unique identifier preventing duplicate processing.
timestamp	Datetime	Exact chronological marker for the temporal sliding

		window.
source_account	String	Originating node identifier.
destination_account	String	Receiving node identifier.
amount	Float	Value transferred, critical for value-flow retention features.
currency	String	Exchange medium, highlighting potential cross-border risk.
payment_type	String	Transfer mechanism (e.g., ACH, Wire, Cash).
is_laundering	Integer	Binary ground-truth label (0 = benign, 1 = suspicious).

The data quality policy dictates that anomalous or malformed records are not silently dropped. Missing amounts, invalid timestamps, or self-transactions trigger rigorous quality checks, generating exception reports that detail the specific rejection rationale. This strict validation preserves the topological integrity of the resulting transaction graph.

Leakage-Safe Temporal Splitting

In financial crime detection, preventing future-data leakage is paramount. When the machine

learning model scores a transaction occurring at time T , it must only have access to

information and structural topologies that existed at or before T ⁵. Randomly shuffling transactions across a dataset inadvertently grants the model access to future network structures, artificially inflating predictive accuracy and rendering the model useless in a live streaming environment.

The architecture enforces a strict chronological split, allocating the earliest 70% of the timeline to training, the subsequent 15% to validation, and the final 15% to out-of-sample testing. Global graph statistics are never calculated over the entire dataset; instead, vertex statistics and topological metrics are computed incrementally. Furthermore, the evaluation framework includes specialized data splits to assess model robustness: an unseen-pattern split tests the model's ability to generalize to novel laundering topologies, while a hard-negative split

introduces structurally complex but legitimate corporate behaviors (e.g., payroll distributions, merchant settlements) to evaluate the false-positive rejection rate.

Baseline Modeling Architecture

The research advances through a hierarchy of baseline models, progressively increasing in structural complexity to quantify the exact value added by graph-based features.

Model M0: The Rule-Based Baseline

The control group simulates legacy transaction monitoring systems. This model flags transactions based on static thresholds, such as exceptionally high transaction velocities, rapid inflow followed immediately by outflow (pass-through), or excessive short-term fan-out behavior¹. While highly interpretable, M0 typically generates an unsustainable volume of false positives, establishing the baseline precision and recall metrics that the machine learning models must surpass.

Model M1: Transaction-Only Gradient Boosting

The second baseline relies purely on tabular, transaction-level machine learning. An XGBoost algorithm is trained using fundamental attributes: log-transformed transaction amounts, temporal markers (hour, day of the week), and basic historical account statistics (recent transaction counts, cumulative source inflows, and counterparty counts). This model addresses the limits of standard supervised learning when disconnected from broader network topologies.

Feature Group	Specific Engineered Features	Purpose
Transaction Attributes	amount, amount_log, payment_type_encoded	Captures the immediate financial risk of the edge.
Temporal Markers	hour, day_of_week, velocity_1h, velocity_24h	Identifies unusual timing and rapid transaction bursts.
Historical Statistics	source_inflow, destination_outflow, counterparty_count	Evaluates long-term account behavior and deviations.

Model M2: Graph Feature Preprocessor (GFP) Integration

The primary research baseline integrates topological pattern extraction. The financial system is represented as a directed graph where accounts constitute nodes and transactions represent directed edges containing attributes such as amount and timestamp⁴. The architecture integrates an established Graph Feature Preprocessor (GFP) implementation, extracting

localized subgraph motifs.

GFP operates by scanning the transaction stream and incrementally maintaining an in-memory graph. For every transaction, GFP generates pattern histograms representing participation in specific typologies¹¹. These features include fan-in (multiple accounts funneling to one), fan-out (one account distributing to many), simple cycles (funds returning to the originator), and scatter-gather structures (funds dispersing through intermediaries before aggregating)⁹. Furthermore, vertex-statistical features maintain incremental historical profiles for each node, capturing mean, variance, and skewness of inflows and outflows without requiring full historical recalculations⁴. These graph features are concatenated with the M1 tabular features and fed into an XGBoost classifier, establishing the M2 benchmark.

Streaming Graph Construction and Dynamic Memory Management

Transitioning from static datasets to continuous-time dynamic graphs introduces severe computational overhead. The graph must evolve instantaneously as transactions arrive, necessitating real-time edge insertion and eviction⁶. If a model evaluates a transaction on Day 10 using a static network snapshot that includes edges from Day 15, the structural integrity is compromised.

Sliding Temporal Windows

To capture accurate behavioral contexts, FlowGuard utilizes a sliding temporal window

mechanism. When a transaction edge e_{uv} occurs at time t_{uv} , the active graph is restricted to a specific look-back period δ_p . The active edges utilized for feature extraction are defined by the time window $TW = [t_{uv} - \delta_p, t_{uv}]$ ⁹. This ensures that long-dormant historical ties do not artificially influence the immediate risk profile of an account, which is crucial for identifying rapid, short-term smurfing campaigns².

Garbage Collection and Latency Optimization

Maintaining a dynamic sliding window across millions of nodes risks catastrophic memory fragmentation and latency spikes if expired edges are not managed efficiently. Naive recursive decrementing—where edges falling outside the temporal window are instantly deleted, triggering a cascade of memory reallocations—stalls the primary feature extraction thread¹². To maintain deterministic throughput, the graph processor implements a modified mark-and-sweep garbage collection protocol. As new edges are inserted, outdated edges are simply marked as logically deleted (tombstoned) rather than physically removed¹². A separate, concurrent background mutator thread periodically sweeps these tombstoned edges and compacts the adjacency lists¹². This deferred decrementing guarantees that the critical path of real-time feature extraction remains unblocked, ensuring stable inference latency measured in milliseconds¹².

Algorithmic Complexities: Throughput vs. Detection

Efficacy

The selection of the final scoring algorithm requires balancing minority-class detection accuracy with the computational throughput required for enterprise-scale transaction monitoring.

Deep Graph Networks vs. Gradient Boosted Trees

While Graph Neural Networks (GNNs) such as Graph Isomorphism Networks (GIN) or Principal Neighbourhood Aggregation (PNA) represent standard approaches for topological learning, they frequently underperform in highly imbalanced financial datasets⁹. Furthermore, training continuous-time dynamic GNNs requires complex memory modules that induce significant processing latency⁴.

Empirical benchmarks demonstrate that extracting explicit graph motifs via a high-speed preprocessor and passing those tabular features to a gradient-boosted tree (XGBoost or LightGBM) drastically outperforms standard GNNs. For instance, on the AML-HI Small dataset, standard GIN architectures achieve an F1 score of approximately 28.70%, whereas a GFP + XGBoost pipeline achieves 64.77%⁹. Graph Transformers like FraudGT achieve even higher accuracy (76.41%) but suffer from extensive computational overhead, rendering them less viable for real-time, low-latency streaming¹⁰. FlowGuard anchors its architecture on XGBoost, ensuring that the model natively handles the dense feature matrices produced by the graph builder while maintaining high execution speeds.

Multi-Stage Fuzzy Mining (BlazingAML Inspiration)

The primary limitation of the traditional GFP approach is its reliance on exact pattern matching. To evade detection, launderers introduce structural fuzziness (altering the number of intermediary hops) and temporal fuzziness (delaying the integration phase)⁸. Scanning an entire graph for all exact permutations of a fuzzy pattern triggers a combinatorial explosion, severely degrading memory and throughput⁸.

To overcome this, the research trajectory explores multi-stage fuzzy graph mining, heavily inspired by the BlazingAML architecture⁸. Instead of searching for rigid predefined subgraphs, complex laundering schemes are decomposed into logical, independent stages connected by fundamental set operations (e.g., intersections of active neighborhoods)⁸. This high-level pattern logic is compiled directly into optimized C++ and CUDA kernels, leveraging power-law-aware memory access patterns and degree-based workload balancing⁸. This transition from exact subgraph counting to optimized, fuzzy multi-stage mining allows the system to achieve equivalent or superior F1 scores while delivering order-of-magnitude throughput accelerations over CPU-bound preprocessors⁸.

Evaluation Framework and XAI for Regulatory Compliance

Because suspicious transactions typically account for less than 0.2% of total financial volume⁹, standard accuracy metrics are highly misleading. The evaluation framework strictly prioritizes the Precision-Recall Area Under the Curve (PR-AUC), minority-class F1-scores, and recall at

fixed alert budgets. Crucially, algorithmic performance is decoupled into distinct computational layers, independently measuring data loading time, graph construction latency, feature extraction throughput (transactions per second), inference latency, and peak RAM utilization.

Explainable AI (XAI) and FIU-IND Reporting

In the context of the Prevention of Money Laundering Act (PMLA) and the Reserve Bank of India (RBI) directives, model explainability is not an optional feature; it is a strict regulatory mandate¹. Black-box neural networks that output high risk scores without interpretable rationale cannot be defended during regulatory audits⁷. The AML compliance team must file a Suspicious Transaction Report (STR) with the Financial Intelligence Unit - India (FIU-IND) detailing the precise nature of the anomaly¹.

By utilizing XGBoost, FlowGuard seamlessly integrates SHapley Additive exPlanations (SHAP). The explanation engine translates mathematical feature importance into actionable intelligence. Instead of generating a generic alert, the system outputs contextual narratives. The dashboard will explicitly state that a transaction was flagged due to a 300% deviation in short-term transaction velocity combined with active participation in a three-hop cycle pattern and an abnormally high fan-out count². This transparency allows the Principal Officer to validate the algorithmic output against established regulatory typologies, drastically reducing investigation times⁷.

Research Innovation Stage: Advanced Feature Engineering

Following the stabilization of the GFP + XGBoost baseline, the research advances through controlled ablation studies, systematically introducing novel feature spaces to quantify their impact on the PR-AUC and false-positive rates.

Temporal Flow Motifs and ExSTraQt Community Metrics

While existing graph features detect structural presence, they often ignore the nuanced timing of the fund movement. Temporal flow features measure the exact time elapsed between consecutive hops, transaction burst durations, and time-decayed path activity. Drawing from the ExSTraQt framework, the system will utilize bottom-up random-walk algorithms (PageRank variants) to define local overlapping communities¹⁰. By calculating weighted temporal metrics within these localized communities, the model identifies the rapid, coordinated bursts indicative of automated mule-network layering¹⁶.

Value-Flow Features

Topological connections confirm a path exists, but money laundering is fundamentally about the transfer of value. Value-flow features analyze the financial physics of the graph. New metrics include the amount-retention ratio (the percentage of source funds that successfully arrive at the final destination), inflow-to-outflow ratios, and value dissipation metrics across intermediate hops. An edge where 98% of the incoming value is immediately transferred outward triggers a significantly higher anomaly score than one where funds are gradually drawn down, directly targeting pass-through mule typologies¹.

Risk-Adaptive Graph Expansion and Behavioral Novelty

Extracting deep multi-hop features for every benign transaction wastes immense computational resources. Risk-adaptive graph expansion introduces a dynamic exploration threshold. An incoming transaction undergoes an immediate one-hop neighborhood analysis. If the localized risk score remains below a defined threshold, graph traversal halts. If the initial topology flags a potential anomaly, the extraction engine dynamically expands to two-hop or three-hop depths to gather broader context. Simultaneously, behavioral novelty features quantify deviation from a specific account's historical baseline, scoring the sudden appearance of new counterparty clusters or unprecedented payment modalities.

Experiment ID	Feature Combination	Model Architecture	Expected Evaluation Outcome
E1	Transaction features only	XGBoost	Establishes the tabular baseline.
E2	GFP structural features	XGBoost	Establishes the topological baseline.
E3	GFP + Temporal Flow Motifs	XGBoost	Analyzes the impact of timing and burst detection.
E4	GFP + Value-Flow Features	XGBoost	Measures detection uplift on pass-through topologies.
E5	GFP + Behavioral Novelty	XGBoost	Evaluates false-positive reduction on legitimate accounts.
E6	Adaptive Graph Expansion	XGBoost	Assesses the trade-off between throughput and recall.

E7	Optimized Final Ensemble	XGBoost	The finalized FlowGuard architecture.
----	--------------------------	---------	---------------------------------------

System Deployment and Reproducibility Engineering

The finalized FlowGuard architecture transitions into a production-ready demonstration application. The backend is orchestrated via FastAPI, providing distinct endpoints for system health checks, real-time transaction scoring, topological graph retrieval, and SHAP-based explanation generation. The frontend, constructed in Streamlit or React, offers compliance analysts a comprehensive dashboard featuring transaction risk scoring, interactive network visualizations of suspicious paths, and detailed model performance monitoring. Reproducibility dictates the engineering culture. Every algorithmic experiment generates a machine-readable JSON artifact within a lightweight experiment registry (e.g., MLflow), recording the dataset split, feature set, random seeds, Git commit hashes, and the complete suite of performance metrics. Automated testing protocols utilizing Pytest ensure that timestamp sequencing remains valid, edge directions are strictly maintained, and self-loops are handled gracefully during the CTDG construction.

Implementation Timeline and the First Practical Milestone

The execution of the FlowGuard research spans a ten-week structured timeline.

Phase	Core Focus Areas	Deliverables
Week 1	Dataset acquisition, verification, project structure setup.	Raw data layer, generic data loader.
Week 2	Schema canonicalization, EDA, missing value validation.	Quality report, topological distribution metrics.
Week 3	Temporal splits, rule-based baselines, tabular XGBoost.	M0 and M1 baselines, metric tracking framework.
Week 4	CTDG construction, sliding windows, garbage collection.	Streaming graph module, memory utilization benchmarks.

Week 5	GFP integration, exact pattern mining, GFP + XGBoost.	M2 baseline, initial F1 and throughput results.
Week 6	Error analysis, SHAP XAI implementation, latency profiling.	False-positive breakdown, explainability module.
Week 7	Temporal flow, community context, value-flow innovations.	Feature ablation studies.
Week 8	BlazingAML fuzzy mining concepts, risk-adaptive expansion.	Accuracy vs. throughput optimization report.
Week 9	Final model selection, cross-dataset validation (ETH Phishing).	Final FlowGuard architecture frozen.
Week 10	FastAPI backend, Streamlit dashboard, STR agentic workflows.	End-to-end demonstration, final technical report.

The critical directive for this architecture is the enforcement of the **M0 Milestone**. Before any advanced fuzzy mining, adaptive expansion, or agentic workflows are developed, a fully reproducible pipeline must be established. The M0 milestone is achieved only when the raw data is ingested, strictly split by time, constructed into a dynamic graph, evaluated through the basic GFP + XGBoost pipeline, and logged in the experiment registry. This systematic, evidence-based approach ensures that FlowGuard evolves into a mathematically sound, highly scalable, and regulatory-compliant solution for modern financial crime detection.

Future Horizons: Collaborative AML via Fully Homomorphic Encryption

While FlowGuard provides exceptional intra-institutional detection, launderers intentionally fragment their networks across multiple, disconnected banking entities to obscure the global topology¹⁷. Expanding the FlowGuard architecture to encompass multi-bank data sharing introduces severe privacy and data-localization hurdles.

Future iterations of the architecture will integrate Fully Homomorphic Encryption (FHE) over the

Torus (TFHE) utilizing frameworks such as Concrete ML¹⁸. In this collaborative paradigm, participating financial institutions encrypt their localized transaction edges using a shared public key¹⁹. The centralized FlowGuard processor ingests these encrypted structures and executes the graph feature extraction and XGBoost inference directly on the ciphertexts, without ever decrypting the underlying Personally Identifiable Information (PII) or transaction amounts¹⁸.

The resulting encrypted risk scores are returned to the originating institution, which utilizes a localized private key for decryption. This cryptographic advancement allows FlowGuard to achieve holistic, cross-border network visibility, identifying massive, distributed laundering networks while strictly preserving client confidentiality and adhering to international privacy regulations.

Works cited

1. Transaction Monitoring Requirements | KYC Hub, <https://www.kychub.com/blog/transaction-monitoring-requirements>
2. AML Transaction Monitoring in India: RBI, PMLA, FIU-IND - KYC Hub, <https://www.kychub.com/blog/transaction-monitoring-aml-india>
3. KYC and RBI Guidelines: Safeguarding Against Money Laundering, <https://thelaw.institute/business-law-as-applicable-to-co-operative-ii/kyc-rbi-guidelines-against-money-laundering/>
4. Graph Feature Preprocessor: Real-time Subgraph-based ... - arXiv, <https://arxiv.org/pdf/2402.08593>
5. TGLite: A Lightweight Programming Framework for Continuous-Time, <https://charithmendis.com/assets/pdf/24-asplos-tglite.pdf>
6. UTG: Towards a Unified View of Snapshot and Event Based Models, <https://arxiv.org/html/2407.12269v1>
7. Why Ethical AI Governance Is the Next AML Frontier - TrackWizz, <https://trackwizz.com/knowledge-hub/why-ethical-ai-governance-is-the-next-aml-frontier/>
8. BlazingAML: High-Throughput Anti-Money Laundering (AML ... - arXiv, <https://arxiv.org/html/2604.12241v1>
9. Graph Feature Preprocessor: Real-time Subgraph-based ... - arXiv, <https://arxiv.org/html/2402.08593v2>
10. Extracting Money Laundering Transactions from Quasi-Temporal, <https://arxiv.org/pdf/2604.02899>
11. Graph Feature Preprocessor — Snap ML 1.15.6 documentation, https://snapml.readthedocs.io/en/v1.15/graph_preprocessor.html
12. Memory management and garbage collection, <https://www.cs.cornell.edu/courses/cs4120/2026sp/notes.html?id=gc>
13. Mark-and-Sweep: Garbage Collection Algorithm - GeeksforGeeks, <https://www.geeksforgeeks.org/java/mark-and-sweep-garbage-collection-algorithm/>
14. BlazingAML: High-Throughput Anti-Money Laundering (AML ... - arXiv,

- <https://arxiv.org/pdf/2604.12241>
15. Agentic AI in BFSI Financial Crime Compliance | India - Innefu Labs,
<https://innefu.com/what-agentic-ai-could-mean-for-financial-crime-in-bfsi/>
 16. Extracting Money Laundering Transactions from Quasi-Temporal,
<https://arxiv.org/html/2604.02899v1>
 17. Realistic Synthetic Financial Transactions for Anti-Money ... - ar5iv,
<https://ar5iv.labs.arxiv.org/html/2306.16424>
 18. arXiv:2411.02926v2 [cs.CR] 11 Nov 2024, <https://arxiv.org/pdf/2411.02926>
 19. (PDF) Privacy-Preserving Graph-Based Machine Learning with Fully,
https://www.researchgate.net/publication/385559721_Privacy-Preserving_Graph-Based_Machine_Learning_with_Fully_Homomorphic_Encryption_for_Collaborative_Anti-Money_Laundering